

BUILD-API.md

18. Gestion des erreurs

Version : V1

Statut : Référentiel officiel de gestion des erreurs des API de Vitala

18.1 Objectif

Cette section définit les règles communes de gestion des erreurs pour l'ensemble des API de Vitala.

Son objectif est de garantir que toutes les erreurs soient :

- cohérentes ;
- prévisibles ;
- compréhensibles ;
- facilement exploitables par les applications Web, Mobile, Admin et les services internes.

Aucune API ne doit retourner un format d'erreur différent de celui défini dans cette section.

18.2 Principes

Toutes les erreurs doivent respecter les principes suivants :

- cohérence ;
- simplicité ;
- sécurité ;
- traçabilité ;
- documentation.

Les messages retournés aux clients ne doivent jamais divulguer d'informations sensibles sur l'infrastructure ou l'implémentation interne.

18.3 Format standard des erreurs

Toutes les réponses en erreur doivent utiliser une structure JSON uniforme.

Les champs suivants sont obligatoires :

- `success` : indique que la requête a échoué (`false`) ;
- `error` : code métier de l'erreur ;

- `message` : message lisible destiné au client ;
- `request_id` : identifiant unique de la requête pour faciliter le diagnostic ;
- `timestamp` : date et heure de l'erreur.

Des informations complémentaires peuvent être ajoutées si elles sont documentées.

18.4 Catégories d'erreurs

Les erreurs sont classées en plusieurs catégories :

Erreurs de validation

Exemples :

- données invalides ;
 - champ obligatoire manquant ;
 - format incorrect.
-

Erreurs d'authentification

Exemples :

- utilisateur non connecté ;
 - jeton expiré ;
 - jeton invalide.
-

Erreurs d'autorisation

Exemples :

- permissions insuffisantes ;
 - accès refusé ;
 - politique RLS non satisfaite.
-

Erreurs métier

Exemples :

- ressource déjà existante ;
 - état incompatible ;
 - règle métier violée.
-

Erreurs techniques

Exemples :

- indisponibilité d'un service ;
 - timeout ;
 - erreur interne.
-

18.5 Codes HTTP

Les erreurs doivent utiliser les codes HTTP appropriés.

Exemples :

- 400 — Requête invalide
- 401 — Authentification requise
- 403 — Accès interdit
- 404 — Ressource introuvable
- 409 — Conflit
- 422 — Données valides mais non acceptables métier
- 429 — Limite de requêtes dépassée
- 500 — Erreur interne
- 503 — Service temporairement indisponible

Les codes HTTP doivent toujours être cohérents avec la nature de l'erreur.

18.6 Messages d'erreur

Les messages doivent être :

- explicites ;
- compréhensibles ;
- rédigés dans un langage clair ;
- indépendants des détails techniques internes.

Ils doivent permettre au client de comprendre l'action à entreprendre sans exposer des informations sensibles.

18.7 Codes métier

Chaque erreur métier doit posséder un identifiant unique.

Exemples :

- AUTH_INVALID_TOKEN
- AUTH_SESSION_EXPIRED
- USER_NOT_FOUND
- FLASH_ALREADY_PUBLISHED
- MISSION_ALREADY_COMPLETED
- PERMISSION_DENIED

Les codes métier sont stables dans le temps et ne dépendent pas des messages affichés.

18.8 Journalisation

Toutes les erreurs importantes doivent être enregistrées.

Les journaux doivent contenir, lorsque cela est pertinent :

- l'identifiant de la requête ;
- l'utilisateur concerné ;
- le service appelé ;
- le code d'erreur ;
- la date et l'heure ;
- les informations techniques nécessaires au diagnostic.

Les données sensibles ne doivent jamais être enregistrées en clair.

18.9 Traçabilité

Chaque erreur doit pouvoir être reliée à une requête unique grâce au `request_id`.

Cet identifiant facilite :

- le support utilisateur ;
 - les investigations ;
 - les audits ;
 - la supervision.
-

18.10 Gestion côté client

Les applications clientes doivent distinguer :

- les erreurs récupérables (nouvelle tentative possible) ;
- les erreurs définitives (action utilisateur requise) ;
- les erreurs système (attendre ou contacter le support).

Le comportement attendu doit être documenté pour chaque catégorie.

18.11 Tests

Les scénarios d'erreur doivent être testés au même titre que les scénarios de succès.

Les tests doivent vérifier :

- le code HTTP ;
 - le format JSON ;
 - le code métier ;
 - le message ;
 - la conformité avec cette spécification.
-

18.12 Références croisées

Cette section est liée à :

- BUILD-SECURITY.md ;
 - BUILD-API.md (Conventions REST) ;
 - DATABASE-RLS-MATRIX.md ;
 - BUILD-OBSERVABILITY.md (si créé ultérieurement).
-

18.13 Conclusion

La gestion des erreurs constitue un élément fondamental de la qualité des API de Vitala.

En imposant un format uniforme, des codes métier stables et une journalisation systématique, la plateforme garantit une meilleure expérience développeur, une intégration simplifiée entre les applications et une capacité de diagnostic rapide en production.